
Percona Operator for PostgreSQL on Kubernetes

Curated Reference Index — Community &
Non-Official Resources for PostgreSQL Administration

Compiled for	Cash — Lead DBA / PostgreSQL SME
Role	Online researcher, Percona PG on K8s
Date	03 August 2026
Scope	Blogs, papers, forums, repos, playbooks
Document	1 of 2 — Index (companion: Captured Procedures)

How to use this index

This is a research index of **practitioner and community** material for running and administering PostgreSQL with the Percona Operator on Kubernetes. Entries are grouped by how independent they are from Percona, since you asked specifically for non-official sources. Each entry lists what it covers and why it is worth your time.

Framing note — the community is small, the lineage is not

Independent deep-dives on *this specific* operator are relatively thin; one operator survey openly notes Percona's PostgreSQL operator community is still very small. Two things compensate: (1) the Percona operator is a **hard fork of Crunchy Data's PGO**, so the large body of Crunchy PGO + pgBackRest + Patroni knowledge transfers almost directly; and (2) the **Percona Community Forum** is where real day-2 failure modes get worked out in public. Mine both aggressively.

Provenance labels used below

[independent] = third-party author, no Percona affiliation. **[community]** = user-generated (forum/Q&A;). **[repo]** = source/config repository. **[percona-blog]** = Percona's own engineering blog (practitioner-grade, but not the reference docs — included because you asked for playbooks, clearly flagged so you can weight it).

A. Independent comparison & evaluation write-ups

Best for architecture decisions, operator selection, and day-2 framing. None authored by Percona.

Solanica — PostgreSQL on Kubernetes: honest comparison

[independent] solanica.io/resources/compare-postgres-operators

Compares Percona, CloudNativePG, Zalando and StackGres across features, community health, business viability and day-2 operations; states no vendor paid for placement. Positions Percona as an enterprise-grade operator built on the Crunchy PGO foundation. Good single-page decision aid.

Palark tech blog — Kubernetes operators for PostgreSQL (multi-part)

[independent] palark.com/blog/cloudnativepg-and-other-kubernetes-operators-for-postgresql/

Part 1 surveys Stolon, Crunchy, Zalando, KubeDB and StackGres; Part 2 adds CloudNativePG with a consolidated feature-comparison table. Neutral engineering perspective; useful for placing Percona against the field.

Gabriele Bartolini — CloudNativePG and Crunchy PGO: an honest, opinionated comparison

[independent]

gabrielebartolini.it/articles/2026/05/cloudnativepg-and-crunchy-pgo-an-honest-opinionated-comparison/

Written by a CNPG maintainer (openly biased, and says so). The value for you is the deep, informed dissection of **Crunchy PGO** architecture, image design, backup strategy and upgrade model — all of which map onto Percona's fork. Also covers the Snowflake acquisition of Crunchy and what it implies.

simplyblock — Choosing a Kubernetes Postgres Operator

[independent] simplyblock.io/blog/choosing-a-kubernetes-postgres-operator/

Adoption-and-momentum view (GitHub stars, activity). This is the source that flags Percona's operator community as very small — worth citing when you justify leaning on Crunchy-lineage material.

Portworx — Choosing a Kubernetes Operator for PostgreSQL

[independent] portworx.com/blog/choosing-a-kubernetes-operator-for-postgresql/

Storage-layer angle (StorageClass, HostPath, NFS, dynamic provisioning). Useful when you reason about PVCs, snapshots and storage performance under the Percona operator.

pganalyze — 5mins of Postgres E33 (operator choice & major upgrades)

[independent] pganalyze.com/blog/5mins-postgres-kubernetes-operator-handling-major-version-upgrades

Short and sharp. Key takeaway for a DBA: these operators integrate Postgres with Kubernetes but do not tune Postgres for you — cluster-level configuration is still your job.

B. Community troubleshooting — Percona Community Forum

The closest thing to a crowd-sourced troubleshooting knowledge base for this operator: real failures with logs and (often) resolutions. The companion PDF captures several of these in full. Browse the category directly — it is more current than any static guide:

forums.percona.com/c/postgresql/percona-kubernetes-operator-for-postgresql/68

Thread	What it teaches	Path under forums.percona.com
Backup pod failing on operator 2.8.2	container-create failure (OpenShift); recent (2026)	/t/.../40658
PostgreSQL cluster recovery fails	PITR target between full & incremental backup; bootstrap job crash	/t/.../15860
pgBackRest restore / default restores not working	<i>unknown group 'postgres'</i> ; archive-command failures on restore	/t/.../23582
pgBackRest problem with operator 1.4.1	S3 backups worked on 1.3.0, broke after operator upgrade	/t/.../21809
V2.2 stanza not created (single master)	<i>duplicate options ('repo1-path','repo-path')</i> in [global]	/t/.../24267
Restore not working in Everest	<i>" must be >= 1 character for 'set' option</i>	/t/.../31965
Patroni + pgBackRest full restore problem	2-node Patroni + 3-node etcd; restore mechanics	/t/.../28077
Operator pods fail to come up	RBAC / pgo-target-role issues in a custom namespace	/t/.../19614
Backup fails after helm upgrade	regression pattern after chart bump	/t/.../34775
postgres-stanza-create failed to run	stanza-create bootstrap failures	/t/.../16898

Search technique

Search the forum by **error signature**, not by symptom — e.g. paste the literal [031] / [039] pgBackRest exit codes or the exact WARN string. The operator wraps pgBackRest and Patroni, so most backup/restore errors are really pgBackRest errors and match upstream discussion too.

C. Source & configuration repositories

[percona/percona-postgresql-operator](#)

[repo] github.com/percona/percona-postgresql-operator

Canonical manifests live in `deploy/` — `bundle.yaml`, `cr.yaml`, `backup.yaml`, `restore.yaml`. The **Releases** page doubles as an upgrade-behaviour changelog (v3.0.0 hard fork, PVC-snapshot backups, GA major-upgrade flow, PG 18 default).

percona/percona-helm-charts

[repo] github.com/percona/percona-helm-charts

pg-operator and pg-db charts. The chart `values.yaml` is your full parameter reference for GitOps. OCI artifacts are also published to `ghcr.io/percona/percona-helm-charts`.

ArtifactHub — pg-operator chart

[repo] artifacthub.io/packages/helm/percona/pg-operator

Rendered configurable-parameters table. Note the explicit warning: v2 is not directly compatible with v1 — always pin `--version` to avoid an accidental major jump.

spron-in/blog-data — GitOps/ArgoCD manifests

[repo] github.com/spron-in/blog-data/tree/master/gitops-argocd-postgresql

The actual bundle `.yaml` + `argo-test.yaml` behind the ArgoCD walkthrough (server-side-apply annotations, sync-waves). Directly reusable for your ArgoCD stack.

D. Practitioner playbooks (Percona engineering blog)

Provenance

These are Percona's **blog**, not the reference docs. They are the most step-by-step operational guides available and several map onto your ArgoCD / automation work, so they earn a place — but weight them as vendor-authored. The companion PDF captures the two migration playbooks and the ArgoCD guide in full, restructured as operator-neutral procedures.

Deploy PostgreSQL on Kubernetes using GitOps and ArgoCD

[percona-blog] percona.com/blog/deploy-postgresql-on-kubernetes-using-gitops-and-argocd/

Real gotcha: Percona CRDs exceed Kubernetes' 262144-byte annotation limit on client-side apply, so you must use **ServerSideApply**; plus ArgoCD **sync-waves** to order operator-before-CR (and reverse on delete). Captured in full in PDF 2.

Migrate Crunchy → Percona operator (3-part series)

[percona-blog]

percona.com/blog/migrate-from-crunchy-data-to-percona-postgresql-operator-backup-restore-pv-reuse/

Three tested, reversible migration paths: **standby cluster** (near-zero downtime), **backup-and-restore** (simplest), and **PV reuse** (skip the data copy). This is the strongest operations-playbook content found. Captured in full in PDF 2.

Automated major version upgrades

[percona-blog] percona.com/blog/automated-major-version-upgrades-in-percona-operator-for-postgresql/

Explains the PerconaPGUpgrade CR internals: annotation → pause cluster → multi-version image runs `pg_upgrade` on the PVCs → auto full backup after. Note: previous backups only restore to the *old* version, so a fresh backup post-upgrade is mandatory.

Migrating PostgreSQL to Kubernetes (pgBackRest cutover)

[percona-blog] percona.com/blog/migrating-postgresql-to-kubernetes/

Standby-mode target syncing from an object-storage WAL archive via pgBackRest, then cut over. Good for on-prem/cloud → K8s moves with a small downtime budget.

Run PostgreSQL on Kubernetes: benchmarks & best practices

[percona-blog] percona.com/blog/run-postgresql-on-kubernetes-a-practical-guide-with-benchmarks-best-practices/

Storage-class benchmark data and pitfalls. Pairs well with the Portworx piece when you argue storage choices.

Deploy with Pulumi (IaC)

[percona-blog] percona.com/blog/run-postgresql-on-kubernetes-with-percona-operator-pulumi/

End-to-end GKE + operator + cluster provisioning in code; a Pulumi analogue to your Terraform interest.

E. Transferable lineage — Crunchy Data PGO

Because the Percona operator is a Crunchy PGO fork, Crunchy's own troubleshooting docs and their pgBackRest / Patroni material explain a large share of Percona operator behaviour (backup stanzas, restore jobs, failover). When a Percona-specific answer does not exist, the PGO equivalent usually does. Keep this in your research index; note Crunchy was acquired by Snowflake in 2025, which is shifting where that content lives and how openly it is licensed.

Cross-reference map (labels you'll see in logs)

Percona CRs still carry Crunchy label keys such as `postgres-operator.crunchydata.com/cluster`, `/role=master|primary`, and `/data=postgres`. That is expected in a fork — selectors in your `kubectl` commands and in community PGO answers apply directly to Percona clusters.

Companion document: **PDF 2 — Captured Procedures & Error Signatures** holds the full ArgoCD, backup-restore and PV-reuse playbooks plus annotated forum incidents, written as ready-to-run operational notes.